Mitigating Adversarial Attacks in Federated Learning with Trusted Execution Environments

Simon Queyrut, Valerio Schiavoni, Pascal Felber
Institute of Computer Science (IIUN)
University of Neuchâtel, Neuchâtel, Switzerland
first.last@unine.ch

2023 IEEE 43rd International Conference on Distributed Computing Systems (ICDCS) | 979-8-3503-3986-4/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDCS57875.2023.00069

***Abstract*—The main premise of federated learning (FL) is that machine learning model updates are computed locally to preserve user data privacy. This approach avoids by design user data to ever leave the perimeter of their device. Once the updates aggregated, the model is broadcast to all nodes in the federation. However, without proper defenses, compromised nodes can probe the model inside their local memory in search for adversarial examples, which can lead to dangerous real-world scenarios. For instance, in image-based applications, adversarial examples consist of images slightly perturbed to the human eye getting misclassified by the local model. These adversarial images are then later presented to a victim node's counterpart model to replay the attack. Typical examples harness dissemination strategies such as altered traffic signs (patch attacks) no longer recognized by autonomous vehicles or seemingly unaltered samples that poison the local dataset of the FL scheme to undermine its robustness. PELTA is a novel shielding mechanism leveraging Trusted Execution Environments (TEEs) that reduce the ability of attackers to craft adversarial samples. PELTA masks inside the TEE the first part of the back-propagation chain rule, typically exploited by attackers to craft the malicious samples. We evaluate PELTA on state-of-the-art accurate models using three well-established datasets: CIFAR-10, CIFAR-100 and ImageNet. We show the effectiveness of PELTA in mitigating six white-box state-of-the-art adversarial attacks, such as Projected Gradient Descent, Momentum Iterative Method, Auto Projected Gradient Descent, the Carlini & Wagner attack. In particular, PELTA constitutes the first attempt at defending an ensemble model against the Self-Attention Gradient attack to the best of our knowledge. Our code is available to the research community at https://github.com/queyrusi/Pelta.**

I. INTRODUCTION

The proliferation of edge devices and small-scale local servers available off-the-shelf nowadays generated an astonishing trove of data, to be used in several areas, including smart homes, e-health, *etc*. For several of these scenarios, the data being generated is highly sensitive. While the deployment of data-driven machine learning (ML) algorithms to train models over such data is becoming prevalent, one must take special care to prevent privacy leaks. In fact, it has been shown how, without proper mitigation mechanisms, sensitive data (*i.e*., the one used by such ML during training) can be reconstructed. To overcome this problem, an increasingly popular approach is federated learning (FL) [1], [2]. FL is a decentralized machine learning paradigm, where clients share with a trusted server only their local individual updates, rather than the data used to train it, hence protecting by design the privacy of user data. The trusted FL server is known by all nodes. His role is to build a global model by aggregating the updates sent by the nodes. Once aggregated, the server broadcasts back the updated model to all clients. The nodes will update their models locally and use the following updates with a fresh batch of local data (*i.e*., for inference purposes). This approach prevents user-data from leaving the user devices, as only the local model updates are sent outside the device.

A popular model trained in FL is the transformer [3], a widespread multi-purpose deep learning (DL) architecture. Transformers create a rich and high-performing continuous representation of the whole input (*i.e*., text or images [4]), effectively used across a diverse range of tasks, yielding state-of-the-art results in several areas, *e.g*., language modeling [5], [6], translation [7], audio classification [8], computer vision tasks such as image classification with vision transformer models (ViT) [9], object detection [10], semantic segmentation [11], *etc*. They harness the *self-attention mechanism* [3], which weights the relative positions of tokens inside a single input sequence to compute a representation of that given sequence.

In the FL training context, both the fine tuning of transformer-based models pre-trained on large-scale data and the training of their lightweight counterparts such as Mobile-ViT [12] have sparked interest in industry and academia [13]. Because this family of models demands considerable efforts in design and computing resources, protecting its integrity during a collaborative training requires special attention.

Consider now the following conjectured scenario. A server broadcasts its expensive model to collaborating clients to have it finetuned over their local private data. One of the clients taps into its RAM, copies the model and computes malicious patches, specifically designed to trick the model. Without ever altering the model, he can now run a patch attack [14]: he puts adversarial stickers on objects (roadsigns for instance) that are subject to regular inferences by the FL model: the objects are then misclassified by unaware agents running the collaboratively learned model and an accident may ensue.

Alternatively, the malicious agent initiates a poisoning attack that can break a model's robustness by sending the central server updates that stem from inference on samples engineered with a trojan trigger to create an unsuspected backdoor [15] that can be activated at an inconvenient time by unaware users of the FL model. Similarly, malicious clients can have the model purposefully and repeatedly misclassify their newfound adversarial examples to severely undermine the quality of the

2575-8411/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDCS57875.2023.00069

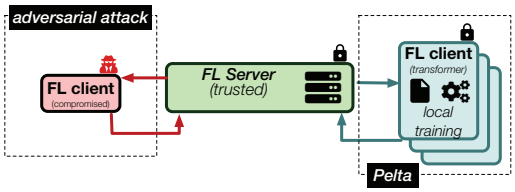

Fig. 1: Federated learning under trusted and compromised nodes. PELTA shields against evasion attacks.

aggregated updates [16]. In all these scenarios, the malicious client accessing its own physical copy of the model allowed him to generate poisonous data that effectively compromised the FL model's reliability. Safeguarding against the creation of these adversaries is paramount in a framework where scaling amplifies the effectiveness of attacks to alarming levels.

In this work, we focus on a client crafting adversarial examples. Fig. 1 depicts our considered FL scenario with a compromised node which tries to craft adversarial samples (*i.e.*, launching an evasion attack). Because it is the hardest to defend against, we consider the *white-box* setting, *i.e.*, the model's characteristics are completely known, and an attacker can leverage gradient computation inside the model to craft adversarial examples. For instance, in the case of vision classification, an attacker leverages the model's gradients to craft images designed to fool a classifier while appearing seemingly unaltered to humans, launching a so-called gradient-based *adversarial* (or *evasion*) *attack* [17]–[19].[1] By design of the FL paradigm, a large number of compromised client can probe their own device memory to launch an adversarial attack against the FL model. In a white-box scenario, these attacks exploit the model in clear at inference time, by perturbing the input and having it misclassified. Such perturbations are typically an additive mask crafted by following the collected gradients of the loss *w.r.t* the input pixels and applying it to the input signal. Such gradients are either directly obtained by tapping into the device's memory when they are effectively computed for local usage or they can be calculated knowing model weights and activations when the device is instructed not to produce them (this is typically the case when running inferences after deployment).

In this paper, we propose PELTA: it is a defense that mitigates gradient-based adversarial attacks launched by a client node by leveraging hardware obfuscation at inference time to hide a few in-memory values, *i.e.*, those close to the input and produced by the model during each pass. This leaves the attackers unable to complete the chain-rule of the back-propagation algorithm used by gradient-based attacks and compute the perturbation to update his adversarial sample. To this end, we rely on hardware-enabled trusted execution environments (TEE), by means of enclaves, secure areas of a processor offering privacy and integrity guarantees. Notable examples include Intel SGX [20] or Arm TrustZone [21]. TEEs can lead to a restriced white-box scenario, *i.e.*, a stricter setting in which the attacker cannot access absolutely everything from the model he seeks to defeat, hence impairing his white-box attack protocols designed for the looser hypothesis. In our context, we deal specifically with TrustZone, given its vast adoption, performance [21] and support for attestation [22]. However, TrustZone enclaves have limited memory (up to 30 MB in some scenarios), making it challenging to completely shield the state-of-the-art Transformer architectures often larger than 500 MB. This constraint therefore calls for such a hardware defense to be a light, partial obfuscation of the model.

Our main contributions are as follows:

(i) We show that it is possible to mitigate evasion attacks during inference in a FL context through hardware shielding.
(ii) We describe the operating principles of PELTA, our lightweight gradient masking defense scheme.
(iii) We apply PELTA to shield layers of individual and ensemble state-of-the-art models against several white-box attack including the Self-Attention Gradient Attack to show the scheme effectively provides high protection in a nigh white-box setting.
(iv) To the best of our knowledge, PELTA is the first applied defense against the Self-Attention Gradient Attack.

The rest of the paper is as follows. §II surveys related work. §III presents our threat model. §IV describes the principles of the PELTA shielding defense. We evaluate it on an ensemble model against a gradient-based attack and discuss the results in §V. §VI discusses general system implications of PELTA. Finally, we conclude and hint at future work in §VII.

II. RELATED WORK

A significant body of work exists towards defending against adversarial attacks in a white-box context [23]. Because of its few-constraints hypothesis, particular endeavour is directed towards refining adversarial training (AT) methods [24], [25]. However, recent studies show a trade-off between a model's generalization capabilities (*i.e.*, its standard test accuracy) and its robust accuracy [26]–[29]. AT can also expose the model to new threats [30] and, perhaps even more remarkably, increase robust error at times [31]. It is possible to use AT at training time as a defense in the federated learning context [32] but it creates its own sensitive exposure to a potentially malicious server. The latter can restore a feature vector that is tightly approximated when the probability assigned to the ground-truth class is low (which is the case for an adversarial example) [33]. Thus, the server may reconstruct adversarial samples of its client nodes, successfully allowing for privacy breach through an *inversion attack*, *i.e.*, reconstructing elements of private data in other nodes' devices. Surprisingly, [32] also uses randomisation at inference time [34] to defend against iterative gradient-based adversarial attacks, even though much earlier work expresses worrying reserves about such practice [35].

[1]While we use Vision Transformers for illustration and description purposes, the mitigation mechanisms presented and implemented later are directly applicable to other classes of models including DNNs or Transformers, such as those for NLP, audio processing, *etc*.

In FL, where privacy of the users is paramount, defending against inversion is not on par with the security of the model in itself and attempts at bridging the two are ongoing [36]. These attempts focus on defending the model at training time or against *poisoning*, *i.e.*, altering the model's parameters to have it underperfom in its primary task or overperform in a secondary task unbeknownst to the server or the nodes. On the other hand, defenses against inversion attacks have seen a surge since the introduction of FL. DarkneTZ [37], PPFL [38] and GradSec [39] mitigate these attacks with the use of a TEE. By protecting sensitive parameters, activations and gradients inside the enclave memory, the risk of gradient leakage is alleviated and the white-box setting is effectively limited, thus weakening the threat. However, PELTA is conceptually different from these methods, as it does not consider the gradient of the loss with respect to the *parameters*, but with respect to the *input image* instead. The former can be revealing private training data samples in the course of an inversion attack, while the latter only ever directs the perturbation applied to an input when conducting an adversarial attack. Other enclave-based defenses do not focus on adversarial attacks, are based on SGX (meaning looser enclave constraints) and do not deal with ML computational graphs in general [40], [41], yet present defense architectures of layers that fit our case [42]. Where those were tested only for CNN-based architectures or simple DNNs, PELTA can protect a larger, more accurate Transformer-based architecture. The robustness of the two types against adversarial attacks was compared in prior studies [43]–[45].

In [46], authors mitigate inference time evasion attacks by pairing the distributed model with a node-specific *attractor*, which detects adversarial perturbations, before distributing it. However, [46] assumes the nodes only have black-box access to their local model. Given the current limitations on the size of encrypted memory in particular for TrustZone enclaves [21], it is currently unfeasible to completely shield models such as VGG-16 or larger. This is why PELTA aims at exploring a more reasonable use case of the hardware obfuscation features of TrustZone by shielding only a subset of the total layers, as we detail more later. It could thus be used as an overlay to the aforementioned study. More generally, our proposed defense scheme does not interfere with existing software solutions for train time or inference time defenses such as randomization, quantization or encoding techniques [47]. As a result, PELTA should not be regarded as a competitor algorithm when it comes to obfuscating sensitive gradients but rather as a supplementary hardware-reliant aid to existing protocols.

Overall, gradient obfuscations as a defense mechanism against evasion attacks have been studied in the past [35], [48]. Authors discuss from a theoretical perspective the fragility of relying on masking techniques. Instead, we show that it fares well protecting a state-of-the-art architecture against inference time gradient-based evasion attacks even by using off-the-shelf hardware with limited resources. We further elaborate on the strong ties to the Backward Pass Differentiable Approximation (BPDA) attack [35] in §IV. While we show that PELTA unambiguously weakens a malicious agent in the white-box setting by restricting their lever of performance, its design provides no defense capabilities against black-box attacks [49] since they operate in a setting that already assumes complete obfuscation of the model's quantities for crafting the adversarial samples.

III. THREAT MODEL

We assume an honest-but-curious client attacker, which does not tamper with the FL process and message flow. The attacker's device is assumed to be computing the gradients of the model at inference time, which is typically the case at each inference during the training rounds of a FL scheme. The case where no gradients are produced/stored by the device is a subcase of this setting. The normal message exchanges defined by the protocol are not altered. The attacker has access to the model to run inferences, but a subset of the layers are *shielded*, under a restricted white-box setting ensured by an impregnable TEE enclave (side channel attacks are out of scope for the rest of this paper). In practice, secure communication channels are established so that only privileged users can allow data recovery from within the TEE. A layer l is shielded (as opposed to its normal *clear* state) if some variables and operations that directly lead to computing gradient values from this layer are obfuscated, *i.e.*, hidden from an attacker. In a typical deep neural network (DNN) f such that

$$f = \mathrm{softmax} \circ f^n \circ f^{n-1} \circ \cdots \circ f^1$$

with layer $f^i = \sigma^i(W^i \cdot x + b^i)$ (σ^i are activation functions), this implies, from shallow (close to the input) to deep (close to loss computation): the input of the layer a^{l-1}; its weight W^l and bias b^l; its output z^l and parametric transform a^l. In general terms, a layer encompasses at least one or two transformations. The attacker probes its own local copy of the model to search for adversarial examples, ultimately to present those to victim nodes and replicate the misclassification by their own copy of the model.

IV. DESIGN

We explain first the design of PELTA shielding of a model, with details on the general mechanisms and the shielding approach in §IV-B. Then, we confront PELTA to the case of the BPDA [35] attack in §IV-C.

A. Back-Propagation Principles and Limits

Recall the back-propagation mechanism; consider a computational graph node x, its immediate (*i.e.*, generation 1) children nodes $\{u_j^1\}$, and the gradients of the loss function with respect to the children nodes $\{d\mathcal{L}/du_j^1\}$. The back-propagation algorithm uses the chain rule to calculate the gradient of the loss function $\mathcal{L}$ with respect to x as in

$$\frac{d\mathcal{L}}{dx} = \sum_j \left(\frac{\partial f_j^1}{\partial x}\right)^T \frac{d\mathcal{L}}{du_j^1} \tag{1}$$

where f_j^1 denotes the function computing node u_j^1 from its parents α^1, *i.e.*, $u_j^1 = f_j^1(\alpha^1)$. In the case of Transformer

encoders, f_j^i can be a convolution, a feedforward layer, an attention layer, a layer-normalization step, *etc*. Existing shielded white-box scenarios [50] protect components of the gradient of the loss *w.r.t.* the parameters $\nabla_\theta \mathcal{L}$, to prevent leakage otherwise enabling an attacker to conduct inversion attacks. Instead, we seek to mask gradients that serve the calculation of the gradient of the loss *w.r.t.* the *input* $\nabla_x \mathcal{L}$ to prevent leakage enabling gradient-based adversarial attacks, as those exploit the backward pass quantities (*i.e*., the gradients) of the input to maximise the model's loss.

Because the model shared between nodes in the FL group still needs to back-propagate correct gradients at each node of the computational graph, masking only an intermediate layer is useless, since it does not prevent the attacker from accessing the correct in-memory gradients on the clear left-hand (shallow) side of the shielded layer. We thus always perform the gradient obfuscation of the first trainable parameters of the model, *i.e*., its shallowest successive layers: it is the lightest way to alter $\nabla_x \mathcal{L}$ through physical masking. Specifically, using Eq. 1, where x denotes the input image (*i.e*., the adversarial instance x_{adv}), an attacker could perform any gradient-based evasion attack (a special case where a layer is not differentiable is discussed in §IV-C). PELTA renders the chain rule incomplete, by forcibly and partially masking the left-hand side term inside the sum, $\{\partial f_j^1/\partial x\}$, hence relying on the lightest possible obfuscation to prevent collecting these gradients and launch the attack.

B. PELTA *shielding*

We consider the computational graph of an ML model:

$$\mathcal{G} = \langle n, l, E, u^1 \dots u^n, f^{l+1} \dots f^n \rangle$$

where n and l respectively represent the number of nodes, *i.e*., transformations, and the number of leaf nodes (inputs and parameters) *s.t.* $1 \leq l < n$. E denotes the set of edges in the graph. For each (j, i) of E, we have that $j < i$ with $j \in \{1...(n-1)\}$ and $i \in \{(l+1)...n\}$. The u^i are the variables associated with every numbered vertex i inside $\mathcal{G}$. They can be scalars or vector values of any dimension. The f^i are the differentiable functions associated with each non-leaf vertex. We also assume a TEE enclave $\mathcal{E}$ that can physically and unequivocally hide quantities stored inside (*e.g*., side-channel to TEEs are out of scope).

To mitigate the adversarial attack when gradients are stored in memory, the PELTA shielding scheme presented in Alg. 1 stores in the TEE enclave $\mathcal{E}$ all the children jacobian matrices of the first layer, $\{\partial f_j^1/\partial x\}$. Note that, given the summation in Eq. 1, obfuscating only some of the children jacobians $\partial f_1^1/\partial x$, $\partial f_2^1/\partial x$... would already constitute an alteration of $\nabla_x \mathcal{L}$. We however chose that PELTA masks all the partial factors of this summation to not take any chances. This obfuscation implies that intermediate gradients should be masked as well. Indeed, because they lead to $\partial f_j^1/\partial x$ through the chain rule, the intermediate gradients (or *local jacobian*) $J^{0\to 1} = \partial f^1\left(\alpha^1\right)/\partial u^0$ between the input $x = u^0$ of the ML

Algorithm 1: PELTA shielding

Data: $\mathcal{G} = \langle n, l, E, u^1 \dots u^n, f^{l+1} \dots f^n \rangle$; enclave $\mathcal{E}$

Algorithm PELTA($\mathcal{G}$)

1 $S \leftarrow$ Select($u^{l+1} \dots u^n$)

2 **for** u *in* S **do** Shield($u, \mathcal{E}$)

3 **return**

Algorithm Shield($u^i, \mathcal{E}$)

4 $\mathcal{E} \leftarrow \mathcal{E} + \{u^i\}$

5 $\alpha^i \leftarrow \langle u^j \mid (j, i) \in E \rangle$ // get parent vertices

6 **for** u^j *in* α^i **do**

7 **if** u_j *is input* **then**

8 $J^{j\to i} \leftarrow \partial f^i\left(\alpha^i\right)/\partial u^j$ // local jacobian

9 $\mathcal{E} \leftarrow \mathcal{E} + \{J^{j\to i}\}$

10 Shield($u^j, \mathcal{E}$) // move on to parent

11 **return**

model and its first transformation must be masked (Alg. 1-line 9). Because one layer may carry several transforms on its local input (*e.g*., linear then ReLU), local jacobians should be masked for at least as many subsequent generations of children nodes as the numbers of layers to shield. The number of generations is directly determined at the arbitrary selection step (Alg. 1-line 1) where the defender choses how far the model should be shielded, *i.e*., the deepest masked nodes. In practice, selecting the first couple of nodes likely induces enough alteration to mitigate the attack and prevent immediate reconstruction of the hidden parameters through either direct calculus (input-output comparison) or inference through repeated queries [51]. From this deep frontier, the defender may recursively mask the parent jacobians (Alg. 1-line 9, 11). Note that this step is skipped in practice when the device doesn't store any gradients.

In adversarial attacks, the malicious user keeps the model intact and treats only the input x_{adv} as a trainable parameter to maximize the prediction error of the sample. We note that the local jacobians (Alg. 1-line 8) between a child node and their non-input parents need not be hidden because the parents are not trainable (they should be input of the model to meet the condition at Alg. 1-line 7). Such quantities are, in effect, simply not present in the back-propagation graph and could not constitute a leak of the sensitive gradient. Similarly, notice how SELECT supposes the deepest masked nodes be chosen *s.t.* they come *after* every input leaf nodes (*i.e*., from subsequent generations). This condition: $u^i \in S \Rightarrow i > l$, insures no information leaks towards trainable input leaf nodes. After all sensitive partials are masked by Alg. 1, such a set of obfuscated gradients is a subset of the total forward partials that would lead to a complete chain rule and is noted $\{\partial f/\partial x\}^L$ assuming $L \leq n$ is the deepest vertex number that denies the attacker information to complete the chain rule. Since all the forward partials down to L are masked, then $\partial f^L/\partial x$ is protected and the resulting under-factored gradient vector (*i.e*., the *adjoint* of f^{L+1}) is a vector in the shape of the shallowest clear layer f^{L+1} and noted $\delta_{L+1} = d\mathcal{L}/du^{L+1}$.

A white-box setting assumes an attacker has knowledge of both the subset $\{\partial f/\partial x\}^L$ of all forward partials and δ_{L+1} to perform a regular gradient-based update on his x_{adv}. However in PELTA, the attacker is only left with the adjoint δ_{L+1}.

Finally, the forward pass quantities u^i that may lead to the unambiguous recovery of the hidden set $\{\partial f/\partial x\}^L$ are masked (Alg. 1-4). This insures that arguments α^i of the transformations f^i cannot be further exploited by the attacker. This could happen when one node is a linear transform of the other as in $u^{i+1} = f^{i+1}((W, u^i)) = W \times u^i$. In this case, the local jacobian $J^{i \to i+1}$ is known to be exactly equal to W. Notice that weights and biases of a DNN would be effectively masked, as they are regarded as leaf vertices of the model's computational graph for the $f^i((u^i, u^{i-1}, u^{i-2})) = u^{i-1} \cdot u^i + u^{i-2} = Wx + b$ operation. Similarly, the outputs of the transformations, $u^i = f^i(\alpha^i)$, are masked by (Alg. 1-line 4). As an example, for a DNN, the exact quantities enumerated in §III are stored in the enclave $\mathcal{E}$.

Overall, PELTA should be construed as simply the general principle of unequivocally hiding enough parameters and gradients that are directly next to the input so that they cannot be maliciously exploited.

C. Relation with BPDA

When training a DL model, a straight-through estimator allows a simple approximation of the gradient of a non-differentiable function (*e.g.*, a threshold) [52]. In a setting discussed in [35], this idea is generalized to a so called Backward Pass Differentiable Approximation (BPDA) to illustrate how preventing a layer from being differentiable in hopes of defeating a gradient-based attack actually constitutes a fragile security measure. In BPDA, the non-differentiable layer f^l of a neural network $f^{1\dots L}(\cdot)$ would be approximated by a neural network g *s.t.* $g(x) \approx f^l(x)$ and be back-propagated through g instead of the non-differentiable transform. This method is what a malicious node adopts against PELTA by upsampling the adjoint of the last clear layer δ_{L+1} to bypass the shielded layers. An illustrative case for a DNN is shown in Fig.2. However, the attacker operates with two limiting factors: *(i)* in a real-world scenario, the attacker possesses limited time and number of passes before the broadcast model becomes obsolete; *(ii)* we hypothesize the attacker does not have priors on the parameters of the first layers of the model, therefore the adversary is effectively left without options for computing the gradient-based update other than training a BPDA of the layer. Although this attack makes a fundamental pitfall of gradient masking techniques, it is worth noting this step becomes increasingly difficult for the attacker as larger parts of the model are hidden from him since it would suppose he has training resources equivalent to that of the FL system. As a side note, we mention that there exist recent defenses against BPDA [53].

In §V, we investigate to what extent an adversarial attack can be mitigated by PELTA and whether the upsampling of the under-factored gradient (which is merely a linear transformation of the correct gradient in the case of the first transformation in many vision models) as a substitute for the masked backward process constitutes a possible last resort for the malicious node.

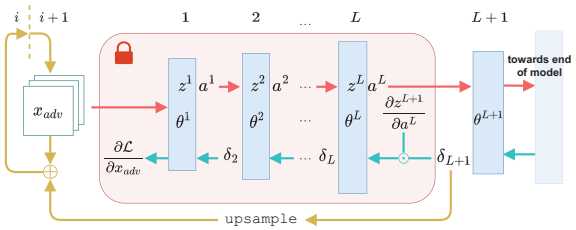

Fig. 2: Overview of the PELTA defense scheme over the first layers of a machine learning model against an iterative gradient-based adversarial attack. Because the attacker cannot access operations in the shallow layers, he resorts to upsample his under-factored gradient δ_{L+1} to compute the adversarial update. The figure depicts activations as transforms for a DNN.

V. EVALUATION

Does PELTA *mitigate white-box attacks?* To answer this question, we conduct attacks on several models with shielded layers.

A. Evaluation Metric and Ensemble Defense Setup

In a common classification task, the *clean accuracy* of a model refers to its standard test accuracy to differentiate it from the context of an adversarial attack where the goal of the defender is to score high on *astuteness* (*i.e.*, *robust* accuracy) against a set of correctly classified samples to which adversarial perturbations were added. This means that a perfectly *astute* model would almost always correctly classify a perturbed sample if he classified it correctly when it was clean. Against adversarial attacks, we chose as defending models several state-of-the-art high clean accuracy models trained on three heavily benchmarked image datasets: CIFAR-10, CIFAR-100 [54] and the ImageNet-21K dataset [55].

1) Individual defenders: Because of the widespread use of the attention mechanism in a large variety of machine learning tasks, we included three size variants of the Vision Transformer in our experiments. Specifically: ViT-L/16, ViT-B/16 and ViT-B/32 [9]. We also included two conventional CNNs, namely ResNet-56, ResNet-164 [56] and two Big Transfer models: BiT-M-R101x3 and BiT-M-R152x4 [57] which stem from the CNN-based ResNet-v2 architecture [56].

2) Ensemble defender: Additionally to these individual defending models we also study the astuteness of an *ensemble* of a ViT and a BiT. An ensemble of models consists of two or more models that determine the correct output through a decision policy. We chose an ensemble because, generally, when dealing with the image classification task, adversarial examples do not *transfer* well between attention based and CNN based models [44]. This means that an example crafted to

Model	Shielded portion	TEE mem. used
ViT-L/16	1.34%	15.16 MB
ViT-B/16	3.61%	11.97 MB
BiT-M-R101x3	4.50e−3%	65.20 KB
BiT-M-R152x4	9.23e−3%	322.14 KB

TABLE I: Estimated enclave memory cost and model portion shielded in each setting. The ensemble value sums both models in the worst case where enclaves are not flushed between evaluation of either of the two models.

fool one type of model in particular will rarely defeat the other, thus highly benefiting the astuteness of the ensemble. This allows for mitigating attacks that target either one of the two specifically, by exploiting a combination of the model outputs to maximize chances of correct prediction. In this paper, we use random selection [58] as a decision policy, where, for each sample, one of the two models is selected at random to evaluate the input at test time.

PELTA Shielding defense of the white-box. To simulate the shielding inside the TEE enclave, we deprive the attacker of the aforementioned quantities (§IV-B) during the individual attacks (V-A1) and during the attack against the ensemble (V-A2). To the best of our knowledge, this is the first ever attempt at mitigating the recent Self-Attention Gradient Attack (see V-B). Against the ensemble, the attacker is deprived of the sensitive quantities of the two models separately, then jointly. For the ViT models, all transforms up to the position embedding [9] are included. This means that the following steps occur inside the enclave: separation of the input into patches x_p^n, projection onto embedding space with embedding matrix E, concatenation with learnable token x_{class} and summation with position embedding matrix E_{pos}:

$$z_0 = \left[x_{\text{class}}\,; x_p^1 E; x_p^2 E; \cdots ; x_p^N E\right] + E_{\text{pos}}$$

For the Big Transfer (BiT) models, the scheme includes the first weight-standardized convolution [57] and its following padding operation. For the ResNets, the first convolution, batch normalization and ReLU activation are masked. Notice that, for all three model types, we obfuscate either two learnable transformations or a non-invertible parametric transformation like weight-standardization, ReLU or MaxPool [59], so the attacker cannot retrieve the obfuscated quantities without uncertainty. Table I reports the estimated overheads of the shield for each setting: the theoretical memory footprints of each secured intermediate activation, weight and gradient as single-precision floating-point numbers were summed and are shown for the ImageNet dataset variants of the models in the worst case where intermediate activations and gradients inside the shield are not flushed after the back-propagation algorithm uses them to complete the pass. Assuming the most resource-intensive case where gradients are produced, the shielding of the ensemble requires less than 16 MB of TEE memory at the very worst, consistent with what typical TrustZone-enabled devices allow [21]. Notice that, because it only ever obfuscates the shallowest parts of a model, PELTA is barely ever affected by the scale of larger and more complex variants, which makes it suitable for a wide variety of state-of-the-art models.

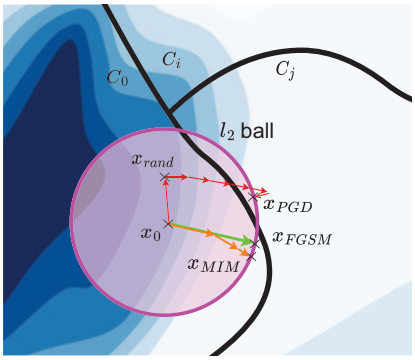

Fig. 3: Schematic diagram of three gradient-based maximum allowable adversarial methods. Within a norm constraint, the attacker computes an additive perturbation of input x_0 to cross a decision boundary of a victim model. Only PGD (red) was able to craft an adversarial example x_{PGD} here.

B. Attacker Setup

Against the individual models (V-A1), we launch four iterative maximum allowable attacks and one regularization-based attack. Against the ensemble model (V-A2) we launch one iterative maximum allowable attack. We shortly introduce all six in their non-targeted version here, *i.e.*, the specific class which the altered sampled is misclassified into has no importance. For the sake of conciseness, we omitted some descriptive equations that can be found in the original papers of the considered attacks.

Iterative maximum allowable attacks (Fig. 3) start at an initial point $x^{(0)}$ which can be chosen as the origin sample x_0 or sometimes as a randomly sampled x_{rand}. The attack then iterates adversarial candidates $x_{adv}^{(i)}$ by following an overall ascendant path of the loss function of the model within a norm constraint. This implies adversarial samples are required to stay inside an l_2 or l_∞ ball centered on x_0, *i.e.*, $||x_{adv}^{(i)} - x_0||_2$ or $||x_{adv}^{(i)} - x_0||_\infty \leq \epsilon, \forall i \geq 0$. In the case of l_∞, this means that the features (in this paper, the individual pixels of the image) cannot vary more than $\pm\epsilon$ in magnitude. Specifically, we use the following attacks in this paper:

Fast Gradient Sign Method - The Fast Gradient Sign Method (FGSM) [17] is a one-step gradient-based approach that finds an adversarial example x_{adv} by adding a single ϵ-perturbation that maximizes the loss function $\mathcal{L}$ to the original sample of label y such that $x_{adv} = x_0 + \epsilon \cdot \text{sign}\left(\nabla_x \mathcal{L}(x_0, y)\right)$.

Projected Gradient Descent - The Projected Gradient Descent (PGD) [60] is the natural multi-step variant of the FGSM algorithm that insures the resulting adversarial example x_{adv} stays within bound of a maximum allowable ϵ-perturbation. This is done through a P operator that projects out of bound values back into the ϵ-ball as illustrated by

the last PGD step of Fig. 3. The i^{th} step is computed as $x^{(i)} = P\left(x^{(i-1)} + \epsilon_{\text{step}} \cdot \text{sign}\left(\nabla_x \mathcal{L}(x_0, y)\right)\right)$, ϵ_{step} being the step size.

Momentum Iterative Method - Inspired by the popular momentum method for accelerating gradient descent algorithms, the Momentum Iterative Method (MIM) [61] applies a velocity vector in the direction of the gradient of the loss function across iterations and at any step $i > 0$ takes into account previous gradients ($g_\mu^{(i)}$ relies on $g_\mu^{(i-1)}$) with a decay factor μ. The additive step is reminiscent of FGSM, as the i^{th} step is computed by $x^{(i)} = x^{(i-1)} + \epsilon_{\text{step}} \cdot \text{sign}(g_\mu^{(i)})$.

Auto Projected Gradient Descent - The Auto Projected Gradient Descent (APGD) [62] proposes adaptive changes of the step size in PGD. This automated scheme allows for an exploration phase and an exploitation phase where an objective function (commonly the cross-entropy loss) is maximized. This attack includes other mechanisms like the ability to restart at a best point along the search. In the benchmark of the individual models, APGD is the most recent attack and it would also be considered the most sophisticated.

We also use a so-called *regularization-based* attack on our individual defending models V-A1:

Carlini and Wagner Attack - The Carlini and Wagner Attack (C&W) [63] iteratively minimizes (typically through the gradient descent algorithm) an objective sum of two competing terms. Through various variable substitutions, one term indirectly measures the norm (typically, l_2) of the added perturbation. On the other hand, a so-called *regularized* term evaluates the wrongness of the classification for an adversarial candidate x_{adv}.

Lastly, we present an iterative gradient-based method against our ensemble defense:

Self-Attention Gradient Attack - To circumvent the ensemble defense V-A2, the attacker uses a gradient-sign attack: the Self-Attention Gradient Attack (SAGA) [44]. It iteratively crafts the adversarial example by following the sign of the gradient of the losses as follows:

$$x^{(i+1)} = x^{(i)} + \epsilon_{\text{step}} * \text{sign}\left(G_{blend}\left(x^{(i)}\right)\right) \tag{2}$$

where we initialize $x^{(0)} = x_0$ the initial image and ϵ_{step} is a given set attack step size (chosen experimentally). Additionally, G_{blend} is defined as a weighted sum, each of the two terms working towards computing an adversarial example against either one of the two models:

$$G_{blend}\left(x^{(i)}\right) = \alpha_k \frac{\partial \mathcal{L}_k}{\partial x^{(i)}} + \alpha_v \phi_v \odot \frac{\partial \mathcal{L}_v}{\partial x^{(i)}} \tag{3}$$

$\partial \mathcal{L}_k / \partial x^{(i)}$ is the partial derivative of the loss of the CNN-based architecture BiT-M-R101x3, and $\partial L_v / \partial x^{(i)}$ is the partial derivative of the loss of the Transformer-based architecture ViT-L/16. Their prominence is controlled by the attacker through two manually set weighting factors, α_k and $\alpha_v = 1 - \alpha_k$. For the ViT gradient, an additional factor is involved: the self-attention map term ϕ_v, defined by a sum-product:

Attack	Parameters (CIFAR-10 and CIFAR-100)
FGSM	ϵ = 0.031
PGD	ϵ = 0.031, ϵ_{step} = 0.00155, steps = 20
MIM	ϵ = 0.031, ϵ_{step} = 0.00155, μ = 1.0
APGD	ϵ = 0.031, N_{restarts} = 1, ρ = 0.75, n^2_{queries} = 5e3
C&W	confidence = 50, ϵ_{step} = 0.00155, steps = 30
SAGA	α_2 = 2.0e−4 and 0.0015, ϵ_{step} = 3.1e−3
Attack	**Parameters (ImageNet)**
FGSM	ϵ = 0.062
PGD	ϵ = 0.062, ϵ_{step} = 0.0031, steps = 20
MIM	ϵ = 0.062, ϵ_{step} = 0.0031, μ = 1.0
APGD	ϵ = 0.062, N_{restarts} = 1, ρ = 0.75, n^2_{queries} = 5e3
C&W	confidence = 50, ϵ_{step} = 0.0031, steps = 30
SAGA	α_k = 0.001 and 0.0015, ϵ_{step} = 0.0031

TABLE II: Attack parameters.

$$\phi_v = \left(\prod_{l=1}^{n_l}\left[\sum_{i=1}^{n_h}\left(0.5 W_{l,i}^{(att)} + 0.5 I\right)\right]\right) \odot x^{(i)} \tag{4}$$

where n_h is the number of attention heads per encoder block of the ViT model, n_l the number of encoder blocks in the ViT model. In the ViT-L/16 for instance, $(n_h, n_l) = (16, 24)$. $W_{l,i}^{(att)}$ is the attention weight matrix in each attention head and I the identity matrix. $\odot$ is the element wise product.

Facing the PELTA shielded setting, the attacker carries out the SAGA without the masked set $\{\partial f / \partial x\}^L$ (and the adjacent quantities otherwise enabling its unambiguous reconstruction). It tries to exploit the adjoint δ_{L+1} of the last clear layer by applying to it a random-uniform initialized upsampling kernel. This process, called *transposed convolution* [64], essentially is a geometrical transformation applied to the vector gradient at the backward pass of a convolutional layer. Although this method does not offer any guarantee of convergence towards a successful adversarial example, it allows to understand whether the adjoint can still serve as a last resort when no priors on the shielded parts are available under limited resource constraint. Individual models are attacked in a similar manner, replacing gradient terms of the shielded computations of the defender by gradients of a substitute transposed convolution. We will therefore ask: in the absence of the shielded quantities, *does upsampling constitute a last resort for the attacker?*

C. Benchmarks and results

We select 1000 correctly classified random samples from CIFAR-10, CIFAR-100 and the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) [55], a subset of ImageNet-21K. This means that the robust accuracy over these samples is 100% if no attack is run. We evaluate the average robust accuracy of individual models against five attacks in a setting where the model is not shielded and a setting where the model is shielded. The ensemble model is evaluated against

CIFAR-10	FGSM		PGD		MIM		C&W		APGD		Clean
ViT-L/16	57.2%	99.3%	1.2%	99.2%	6.2%	98.7%	0.0%	99.1%	0.0%	90.6%	99.4%
ViT-B/16	38.6%	91.2%	0.0%	96.2%	0.3%	97.3%	0.0%	95.8%	0.0%	88.9%	98.5%
ViT-B/32	33.5%	92.8%	2.1%	98.0%	3.3%	97.9%	0.0%	96.9%	0.0%	89.9%	98.0%
ResNet-56	23.9%	75.6%	0.0%	81.3%	0.0%	95.9%	0.0%	95.2%	0.0%	57.1%	93.0%
ResNet-164	30.0%	78.7%	0.0%	82.7%	0.0%	96.3%	0.0%	96.0%	0.0%	60.0%	93.9%
BiT-M-R101x3	84.8%	90.9%	0.0%	74.1%	0.0%	83.1%	0.0%	98.0%	0.0%	57.3%	98.9%
CIFAR-100	**FGSM**		**PGD**		**MIM**		**C&W**		**APGD**		**Clean**
ViT-L/16	30.1%	99.2%	1.0%	98.9%	4.0%	98.0%	0.0%	99.0%	0.0%	90.0%	93.9%
ViT-B/16	19.3%	91.1%	2.7%	93.2%	0.9%	96.9%	0.0%	97.9%	0.0%	88.3%	92.5%
ViT-B/32	20.0%	92.9%	2.9%	92.0%	1.9%	98.4%	0.0%	96.2%	0.0%	89.0%	93.5%
ResNet-56	5.2%	81.5%	0.1%	82.6%	3.3%	95.1%	0.0%	96.0%	0.0%	60.8%	70.0%
ResNet-164	7.9%	83.8%	0.2%	83.7%	3.9%	97.6%	0.0%	94.2%	0.0%	62.1%	76.1%
BiT-M-R101x3	3.3%	85.4%	0.0%	75.7%	0.0%	82.9%	0.0%	98.8%	0.0%	59.8%	90.2%
ImageNet	**FGSM**		**PGD**		**MIM**		**C&W**		**APGD**		**Clean**
ViT-L/16	27.5%	94.0%	0.0%	93.9%	0.0%	97.4%	0.0%	99.3%	0.0%	86.5%	82.7%
ViT-B/16	22.1%	92.1%	0.0%	92.0%	0.0%	97.4%	0.0%	99.1%	0.0%	87.1%	80.0%
BiT-M-R101x3	24.2%	83.8%	0.0%	76.8%	0.0%	83.2%	0.0%	98.2%	0.0%	73.4%	79.3%
BiT-M-R152x4	67.0%	85.8%	0.0%	87.1%	0.0%	93.7%	0.0%	98.0%	0.0%	67.1%	85.1%

TABLE III: Robust accuracy of non-shielded (left) versus shielded (right) individual models against a benchmark of five white-box attacks on CIFAR-10, CIFAR-100 and ImageNet (higher values favor the defender). Clean accuracy over 1000 random validation samples is provided for illustrative purpose.

CIFAR-10	Baseline		Applied Shield			
Model Acc.	**Clean**	**Random**	**None**	**ViT-L/16**	**BiT-M-R101x3**	**Ensemble**
ViT-L/16	99.4%	99.5%	28.1%	99.2%	14.1%	99.5%
BiT-M-R101x3	98.8%	98.8%	25.2%	0.3%	78.9%	98.5%
Ensemble	99.1%	98.9%	27.2%	49.7%	46.4%	98.8%
CIFAR-100	**Baseline**		**Applied Shield**			
Model Acc.	**Clean**	**Random**	**None**	**ViT-L/16**	**BiT-M-R101x3**	**Ensemble**
ViT-L/16	94.0%	99.4%	5.2%	99.6%	4.2%	99.8%
BiT-M-R101x3	89.9%	98.3%	18.3%	0.2%	50.0%	82.2%
Ensemble	92.0%	98.9%	12.2%	49.5%	27.5%	90.8%
ImageNet	**Baseline**		**Applied Shield**			
Model Acc.	**Clean**	**Random**	**None**	**ViT-L/16**	**BiT-M-R101x3**	**Ensemble**
ViT-L/16	82.6%	100.0%	6.3%	99.2%	6.1%	97.5%
BiT-M-R152x4	85.6%	100.0%	15.2%	0.6%	45.5%	76.2%
Ensemble	84.3%	100.0%	10.8%	49.9%	25.8%	87.0%

TABLE IV: Robust accuracy of a shielded ensemble against SAGA on 1000 corretly classified CIFAR-10, CIFAR-100 and ImageNet samples (higher values favor the defender). Baseline values show clean accuracy, astuteness against random uniform attack on the l_∞ ball. Applied Shield values show per-model robust accuracy against different shielding setups.

the SAGA in four settings: no model is shielded, only the BiT model is shielded, only the ViT model is shielded, both models are shielded (maximum protection for the ensemble). For reference, the clean accuracy of the models over 1000 random samples from the validation set of each dataset is provided. Attack parameters are provided in Table II.

Table III shows our results for the individual models against the five attacks and Table IV shows our results for the ensemble model against the SAGA. For illustrative purposes, Fig.4 shows the generated perturbation on one sample in the four settings of the ensemble model against the SAGA.

Does PELTA *mitigate white-box attacks?* We observe that the shielding greatly preserves the astuteness of individual models and of the ensemble, with up to 98.8% robust accuracy for the ensemble (1.2% attack success rate) comparable to random uniform pixel modifications, and up to 99.3% robust accuracy for individual models. It can be noted that, generally, the size of the model has a positive influence on the astuteness after shielding across attacks; however, this effect seems largely overwhelmed by the advantage variants of ViT have over CNN-based models before and after shielding. Additionally, we see that for the ensemble defense, individual

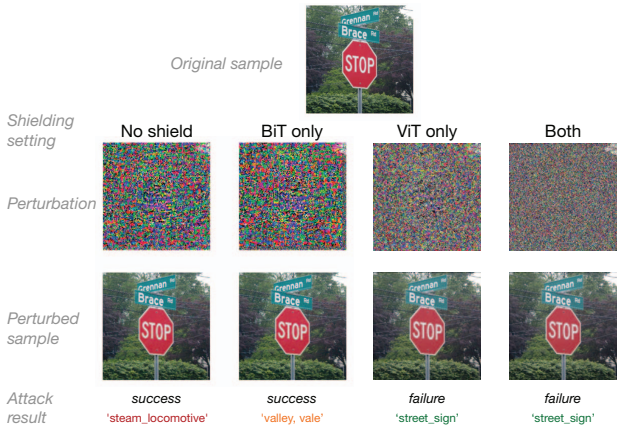

Fig. 4: SAGA adversarial samples in four different shielding settings from a correctly classifed sample.

model robust accuracies are not equally protected: the ViT model benefits more from PELTA when applied only to it than BiT. We explain these results as a general advantage in robustness of Transformer-based architectures over CNNs [43] with BiT being more sensitive to targeted attacks than its counterpart, and also more sensitive to adversarial examples crafted against the ViT. We further note that in the ensemble, individual robust accuracies worsen compared to the non-shielded setting when only their counterpart is shielded. The reason for this is attributed to the fact that SAGA solely directs the sample towards augmenting the non-shielded loss, while disregarding the shielded loss, consequently resulting in the creation of adversarial samples that exclusively aim to exploit vulnerabilities in the non-shielded model.

Does upsampling constitute a last resort for the attacker? Interestingly, for the ensemble, the attack success rate of the upsampling against the PELTA defense scheme sometimes surpasses that of the random uniform attack against BiT when the shield is applied only to it. We explain this behaviour as follows: contrary to the shielded layer in ViT that projects the input onto an embedding space, the last clear layer adjoint δ_{L+1} in the BiT still carries spatial information that could be recovered *e.g.*, through average upsampling. These results suggest shielding both models for optimal security, given sufficient encrypted memory available in a target TEE.

A similar remark can be made about the behavior of CNN-based models in the individual attacks. While ViT variants generally fare very well against most attacks to the exception of the most sophisticated (APGD), ResNets and BiTs have overall low astuteness, indicating the attacks behave largely better than random. This suggests that larger parts of the model should be included in the enclave of the PELTA scheme to mitigate the effectiveness of the upsampling by the attacker.

VI. SYSTEM IMPLICATIONS

As previously mentioned, TEEs typically operate in a secure mode that is designed to provide protection against external attacks. This secure mode can add overhead and complexity to communication between the secure world and the rest of the system, which can in turn impact data throughput. For example, when data is transferred between the TEE and the non-secure world, it may need to be encrypted and decrypted, which can slow down communication. Data transferred between the secure world and the TEE traditionally requires secure communication protocols to prevent unauthorized access or tampering, thus hindering the velocity of the data transfer process, as encryption and decryption may be necessary to protect the data. Moreover, a context switch is typically required to move from one execution context to another. This context switching can introduce additional overhead and slow down the data transfer process.

Because PELTA is designed to hide sensitive data at inference time through the use of a TEE, a throughput bottleneck could be felt at two stages of the federated endeavour. The first case is the most self-explanatory: even after deployment following FL rounds, inference with PELTA still supposes parts of the model are inside a TEE and sensitive operations are carried inside the enclave. This requires context switches and establishing a secure communication channel between worlds to either feed the first computation nodes of the model with the input data or extracting the output of the last shielded layer to carry on subsequent operations with the clear and deeper segment of the model. Fortunately, these elementary TEE methods usually range from microseconds up to milliseconds at most for either SGX or TrustZone [65], [66], which is commensurate with the real-time usage of most current edge ML applications that fit into FL (text prediction, sentiment analysis, health monitoring, speech recognition etc.) as recently demonstrated [67]. As a rule, it should be kept in mind that minimizing context switches is an important optimization technique in the design of TEE applications.

The second case is the training phase of the FL scheme. During this phase, the use of an optimization algorithm puts more strain on the TEE and its communication channels. For instance, inside the enclave, gradients which were not generated during regular end-user inference are now being computed: these gradients seldom need to be read from within the enclave in order to be sent for aggregation, which adds in bandwidth overhead. However, many of these limitations are taken into account when tuning the parameters of the protocol for each FL round. For example, the frequency at which the weight updates are pulled out of the enclave to be sent to the aggregating server could be lowered to allow averaging hidden gradients over larger batches on the client nodes. Overall, training protocols of an FL scheme are expected to harness the idle state of edge devices to handle intermittent compute node availability [68]. The extra bandwidth overhead of the second case should therefore not impact user experience as much as it does the strategy of the FL training rounds.

VII. Conclusion and Future Work

In federated learning, adversarial attacks are the basis of several trojaning and poisoning attacks. However, they are difficult to defend against at inference time under the white-box hypothesis, which is the default setting in FL. We described how to mitigate such attacks by using hardware obfuscation to sidestep the white-box. We introduce a novel defense, PELTA: a lightweight shielding method which relies on TrustZone enclaves to mask few carefully chosen parameters and gradients against iterative gradient-based attacks. Our evaluation shows promising results, sensibly mitigating the effectiveness of state-of-the-art attacks. To the best of our knowledge, PELTA also constitute the first attempt at defending against the Self-Attention Gradient Attack.

We intend to extend this work along the following directions. Because the use of enclaves calls for somewhat costly normal-world to secure-world communication mechanisms, properly evaluating the speed of each collaborating device under our shielding scheme is needed on top of considering the aforementioned memory overheads (Table I) in order to assess the influence of PELTA on the FL training's practical performance. Additionally, a natural extension to this work is to apply PELTA along with existing software defenses [47] to assess their combined benefits against a sophisticated attacker.

As mentioned in §IV-C, it should also be explored that an attacker can *(i)* exploit commonly used embedding matrices and subsequent parameters across existing models as a prior on the shielded layers (this case being circumvented by the defender if it trains its own first parameters) or *(ii)* to train on their own premises the aforementioned g backward approximation (which needs not be of the same architecture as the shielded layers), although recent work shows limitation for such practice [69].

References

[1] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtarik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," in *NIPS Workshop on Private Multi-Party Machine Learning*, 2016.

[2] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[4] A. Raganato and J. Tiedemann, "An analysis of encoder representations in transformer-based machine translation," in *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. The Association for Computational Linguistics, 2018.

[5] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv*, Sep. 2019. [Online]. Available: https://arxiv.org/abs/1909.08053

[6] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

[7] S. Takase and S. Kiyono, "Lessons on parameter sharing across layers in transformers," *CoRR*, vol. abs/2104.06022, 2021. [Online]. Available: https://arxiv.org/abs/2104.06022

[8] A. Nagrani, S. Yang, A. Arnab, A. Jansen, C. Schmid, and C. Sun, "Attention bottlenecks for multimodal fusion," in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 14 200–14 213. [Online]. Available: https://proceedings.neurips.cc/paper/2021/file/76ba9f564ebbc35b1014ac498fafadd0-Paper.pdf

[9] A. Kolesnikov, A. Dosovitskiy, D. Weissenborn, G. Heigold, J. Uszkoreit, L. Beyer, M. Minderer, M. Dehghani, N. Houlsby, S. Gelly, T. Unterthiner, and X. Zhai, "An image is worth 16x16 words: Transformers for image recognition at scale," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*, 2021.

[10] Y. Wei, H. Hu, Z. Xie, Z. Zhang, Y. Cao, J. Bao, D. Chen, and B. Guo, "Contrastive learning rivals masked image modeling in fine-tuning via feature distillation," *Tech Report*, 2022.

[11] H. Bao, L. Dong, S. Piao, and F. Wei, "BEit: BERT pre-training of image transformers," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*, 2022. [Online]. Available: https://openreview.net/forum?id=p-BhZSz59o4

[12] S. Mehta and M. Rastegari, "Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=vh-0sUt8HlG

[13] J. H. Ro, T. Breiner, L. McConnaughey, M. Chen, A. T. Suresh, S. Kumar, and R. Mathews, "Scaling language model size in cross-device federated learning," in *ACL 2022 Workshop on Federated Learning for Natural Language Processing*, 2022. [Online]. Available: https://openreview.net/forum?id=ShNG29KGF-c

[14] T. B. Brown, D. Mané, A. Roy, M. Abadi, and J. Gilmer, "Adversarial patch," *ArXiv*, vol. abs/1712.09665, 2017.

[15] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How To Backdoor Federated Learning," in *International Conference on Artificial Intelligence and Statistics*. PMLR, Jun. 2020, pp. 2938–2948. [Online]. Available: https://proceedings.mlr.press/v108/bagdasaryan20a.html

[16] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing Federated Learning through an Adversarial Lens," in *International Conference on Machine Learning*. PMLR, May 2019, pp. 634–643. [Online]. Available: https://proceedings.mlr.press/v97/bhagoji19a.html

[17] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2015. [Online]. Available: http://arxiv.org/abs/1412.6572

[18] B. Biggio, I. Corona, D. Maiorca, B. Nelson, N. Šrndić, P. Laskov, G. Giacinto, and F. Roli, "Evasion attacks against machine learning at test time," in *Machine Learning and Knowledge Discovery in Databases*, H. Blockeel, K. Kersting, S. Nijssen, and F. Železný, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 387–402.

[19] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. J. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2014. [Online]. Available: http://arxiv.org/abs/1312.6199

[20] V. Costan and S. Devadas, "Intel sgx explained," *Cryptology ePrint Archive*, 2016.

[21] J. Amacher and V. Schiavoni, "On the performance of ARM TrustZone," in *IFIP International Conference on Distributed Applications and Interoperable Systems*. Springer, 2019, pp. 133–151.

[22] J. Ménétrey, M. Pasin, P. Felber, and V. Schiavoni, "WaTZ: A Trusted WebAssembly Runtime Environment with Remote Attestation for TrustZone," in *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2022.

[23] N. Carlini. (2019) A complete list of all (arxiv) adversarial example papers. [Online]. Available: https://nicholas.carlini.com/writing/2019/all-adversarial-example-papers.html

[24] Z. Qian, K. Huang, Q. Wang, and X. Zhang, "A survey of robust adversarial training in pattern recognition: Fundamental, theory, and methodologies," *Pattern Recognit.*, vol. 131, p. 108889, 2022. [Online]. Available: https://doi.org/10.1016/j.patcog.2022.108889

[25] T. Bai, J. Luo, J. Zhao, B. Wen, and Q. Wang, "Recent advances in adversarial training for adversarial robustness," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*, Z.-H. Zhou, Ed. International Joint Conferences on Artificial Intelligence Organization, 8 2021, pp. 4312–4321, survey Track. [Online]. Available: https://doi.org/10.24963/ijcai.2021/591

[26] A. Raghunathan, S. M. Xie, F. Yang, J. Duchi, and P. Liang, "Adversarial training can hurt generalization," in *ICML 2019 Workshop on Identifying and Understanding Deep Learning Phenomena*, 2019. [Online]. Available: https://openreview.net/forum?id=SyxM3J256E

[27] P. Nakkiran, "Adversarial robustness may be at odds with simplicity," *ArXiv*, vol. abs/1901.00532, Jan. 2019.

[28] D. Tsipras, S. Santurkar, L. Engstrom, A. Turner, and A. Madry, "Robustness may be at odds with accuracy," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=SyxAb30cY7

[29] L. Chen, Y. Min, M. Zhang, and A. Karbasi, "More data can expand the generalization gap between adversarially robust and standard models," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*, 2020.

[30] J. Zhang, Y. Chen, and H. Li, "Privacy leakage of adversarial training models in federated learning systems," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 108–114.

[31] J. Clarysse, J. Hörrmann, and F. Yang, "Why adversarial training can hurt robust accuracy," *CoRR*, vol. abs/2203.02006, 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2203.02006

[32] Y. Chang, S. Laridi, Z. Ren, G. Palmer, B. W. Schuller, and M. Fisichella, "Robust federated learning against adversarial attacks for speech emotion recognition," Mar. 2022. [Online]. Available: https://arxiv.org/abs/2203.04696

[33] H. Zhang, H. Chen, Z. Song, D. S. Boning, I. S. Dhillon, and C. Hsieh, "The limitations of adversarial training and the blind-spot attack," in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. [Online]. Available: https://openreview.net/forum?id=HylTBhA5tQ

[34] T. Yu, S. Hu, C. Guo, W.-L. Chao, and K. Q. Weinberger, *A New Defense against Adversarial Images: Turning a Weakness into a Strength*. Red Hook, NY, USA: Curran Associates Inc., 2019.

[35] A. Athalye, N. Carlini, and D. A. Wagner, "Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples," in *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018.*, 2018, pp. 274–283. [Online]. Available: http://proceedings.mlr.press/v80/athalye18a.html

[36] P. Liu, X. Xu, and W. Wang, "Threats, attacks and defenses to federated learning: issues, taxonomy and perspectives," *Cybersecurity*, vol. 5, no. 1, pp. 1–19, 2022.

[37] F. Mo, A. S. Shamsabadi, K. Katevas, S. Demetriou, I. Leontiadis, A. Cavallaro, and H. Haddadi, "Darknetz: Towards model privacy at the edge using trusted execution environments," in *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services*, ser. MobiSys '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 161–174. [Online]. Available: https://doi.org/10.1145/3386901.3388946

[38] F. Mo, H. Haddadi, K. Katevas, E. Marin, D. Perino, and N. Kourtellis, "Ppfl: Privacy-preserving federated learning with trusted execution environments," in *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services*, ser. MobiSys '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 94–108. [Online]. Available: https://doi.org/10.1145/3458864.3466628

[39] A. A. Messaoud, S. B. Mokhtar, V. Nitu, and V. Schiavoni, "Shielding federated learning systems against inference attacks with ARM TrustZone," in *Middleware'22: Proceedings of the 23rd ACM/IFIP International Middleware Conference*. New York, NY, USA: Association for Computing Machinery, Nov. 2022, pp. 335–348.

[40] H. Benkraouda and K. Nahrstedt, "Image reconstruction attacks on distributed machine learning models," in *Proceedings of the 2nd ACM International Workshop on Distributed Machine Learning*, ser. DistributedML '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 29–35. [Online]. Available: https://doi.org/10.1145/3488659.3493779

[41] L. Hanzlik, Y. Zhang, K. Grosse, A. Salem, M. Augustin, M. Backes, and M. Fritz, "MLCapsule: Guarded Offline Deployment of Machine Learning as a Service," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE, Jun. 2021, pp. 3295–3304.

[42] Z. Gu, H. Huang, J. Zhang, D. Su, A. Lamba, D. Pendarakis, and I. M. Molloy, "Securing input data of deep learning inference systems via partitioned enclave execution," *CoRR*, vol. abs/1807.00969, 2018. [Online]. Available: http://arxiv.org/abs/1807.00969

[43] P. Benz, C. Zhang, S. Ham, A. Karjauv, and I. S. Kweon, "Robustness comparison of vision transformer and mlp-mixer to cnns," *Workshop on Adversarial Machine Learning in Real-World Computer Vision Systems and Online Challenges (AML-CV) at CVPR*, 2021.

[44] K. Mahmood, R. Mahmood, and M. van Dijk, "On the robustness of vision transformers to adversarial examples," in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 7818–7827.

[45] A. Aldahdooh, W. Hamidouche, and O. Déforges, "Reveal of vision transformers robustness against adversarial attacks," *CoRR*, vol. abs/2106.03734, 2021. [Online]. Available: https://arxiv.org/abs/2106.03734

[46] J. Zhang, W. J.-W. Tann, and E.-C. Chang, "Mitigating Adversarial Attacks by Distributing Different Copies to Different Users," *arXiv*, Nov. 2021.

[47] K. Ren, T. Zheng, Z. Qin, and X. Liu, "Adversarial attacks and defenses in deep learning," *Engineering*, vol. 6, no. 3, pp. 346–360, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S209580991930503X

[48] F. Tramèr, A. Kurakin, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel, "Ensemble adversarial training: Attacks and defenses," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=rkZvSe-RZ

[49] C. Wang, M. Zhang, J. Zhao, and X. Kuang, "Black-Box Adversarial Attacks on Deep Neural Networks: A Survey," in *2022 4th International Conference on Data Intelligence and Security (ICDIS)*. IEEE, Aug. 2022, pp. 88–93.

[50] A. A. Messaoud, S. B. Mokhtar, V. Nitu, and V. Schiavoni, "Shielding Federated Learning Systems against Inference Attacks with ARM TrustZone," *arXiv*, Aug. 2022.

[51] S. J. Oh, B. Schiele, and M. Fritz, "Towards Reverse-Engineering Black-Box Neural Networks," in *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Cham, Switzerland: Springer, Sep. 2019, pp. 121–144.

[52] Y. Bengio, N. Léonard, and A. C. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *CoRR*, vol. abs/1308.3432, Aug. 2018. [Online]. Available: http://arxiv.org/abs/1308.3432

[53] H. Qiu, Y. Zeng, Q. Zheng, S. Guo, T. Zhang, and H. Li, "An efficient preprocessing-based approach to mitigate advanced adversarial attacks," *IEEE Transactions on Computers*, pp. 1–1, 2021.

[54] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," *Master's thesis, Department of Computer Science, University of Toronto*, 2009.

[55] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet Large Scale Visual Recognition Challenge," *Int. J. Comput. Vision*, vol. 115, no. 3, pp. 211–252, Dec. 2015.

[56] K. He, X. Zhang, S. Ren, and J. Sun, "Identity Mappings in Deep Residual Networks," in *Computer Vision – ECCV 2016*. Cham, Switzerland: Springer, Sep. 2016, pp. 630–645.

[57] A. Kolesnikov, L. Beyer, X. Zhai, J. Puigcerver, J. Yung, S. Gelly, and N. Houlsby, "Big transfer (bit): General visual representation learning," in *Computer Vision – ECCV 2020*, A. Vedaldi, H. Bischof, T. Brox, and J.-M. Frahm, Eds. Cham: Springer International Publishing, 2020, pp. 491–507.

[58] S. Srisakaokul, Y. Zhang, Z. Zhong, W. Yang, T. Xie, and B. Li, "MULDEF: Multi-model-based Defense Against Adversarial Examples for Neural Networks," *arXiv*, Aug. 2018.

[59] M. D. Zeiler and R. Fergus, "Visualizing and Understanding Convolutional Networks," in *Computer Vision – ECCV 2014*. Cham, Switzerland: Springer, 2014, pp. 818–833.

[60] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2018, [Online; accessed 21. Jan. 2023]. [Online]. Available: https://dspace.mit.edu/handle/1721.1/137496?show=full

[61] Y. Dong, F. Liao, T. Pang, H. Su, J. Zhu, X. Hu, and J. Li, "Boosting Adversarial Attacks with Momentum," in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Jun. 2018, pp. 9185–9193.

[62] F. Croce and M. Hein, "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks," in *ICML'20: Proceedings of the 37th International Conference on Machine Learning*. JMLR.org, Jul. 2020, pp. 2206–2216.

[63] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *2017 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2017, pp. 39–57. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP.2017.49

[64] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," *ArXiv*, vol. abs/1603.07285, Mar. 2016.

[65] A. Mukherjee, T. Mishra, T. Chantem, N. Fisher, and R. Gerdes, "Optimized trusted execution for hard real-time applications on COTS processors," in *RTNS '19: Proceedings of the 27th International Conference on Real-Time Networks and Systems*. New York, NY, USA: Association for Computing Machinery, Nov. 2019, pp. 50–60.

[66] O. Weisse, V. Bertacco, and T. Austin, "Regaining Lost Cycles with HotCalls: A Fast Interface for SGX Secure Enclaves," *SIGARCH Comput. Archit. News.*, vol. 45, no. 2, pp. 81–93, Jun. 2017.

[67] M. F. Babar and M. Hasan, "Real-Time Scheduling of TrustZone-enabled DNN Workloads," in *CPSIoTSec '22: Proceedings of the 4th Workshop on CPS & IoT Security and Privacy*. New York, NY, USA: Association for Computing Machinery, Nov. 2022, pp. 63–69.

[68] Y. Yan, C. Niu, Y. Ding, Z. Zheng, F. Wu, G. Chen, S. Tang, and Z. Wu, "Distributed non-convex optimization with sublinear speedup under intermittent client availability," *ArXiv*, vol. abs/2002.07399, 2020.

[69] C. Sitawarin, Z. Golan-Strieb, and D. Wagner, "Demystifying the adversarial robustness of random transformation defenses," in *The AAAI-22 Workshop on Adversarial Machine Learning and Beyond*, 2022. [Online]. Available: https://openreview.net/forum?id=p4SrFydwO5